

C# / .NET Assignment

Part 02 – OOP Concepts

إعداد المتدرب: Mohamed Fouad Helail (محمد فؤاد هليل)

المبادرة: مبادرة رواد مصر الرقمية (DEPI)

إشراف: Eng. Abdelrahman Shaaban

Question 1: LinkedIn Article about Class Types

تعتبر فئات (Class Types) C# من أهم مفاهيم هندسة البرمجيات (Software Design). سأقوم بتوضيح أنواعها المختلفة ومتى نستخدم كل نوع.

1. Normal Class (الفئة العادية)

المعنى: هي الـ **Class** الأساسية التي تعمل كقالب أو مخطط (Blueprint) يمكننا من خلاله إنشاء كائنات جديدة (Objects).

متى نستخدمها: تُستخدم لتمثيل أي كيان في النظام يتطلب إنشاء **Object** خاص به، مثل سيارة أو موظف.

```
class Car
{
    public string Model { get; set; }
}
```

2. Static Class (الفئة الثابتة)

المعنى: هي فئة لا يمكننا أخذ **Object** منها أبداً (Cannot be instantiated). كل محتوياتها يجب أن تكون ثابتة.

متى نستخدمها: نستخدمها لإنشاء دوال مساعدة (Utility/Helper methods) لا تعتمد على حالة كائن معين، مثل العمليات الحسابية.

```
static class MathHelper
{
    public static int Add(int a, int b) => a + b;
}
```

3. Abstract Class (الفئة المجردة)

المعنى: هي فئة ناقصة أو غير مكتملة، لا يمكن أخذ **Object** منها مباشرة. تعمل كـ **Base Class** يجب أن ترث منها فئات أخرى لتكمل تنفيذها.

متى نستخدمها: عندما يكون لدينا مفهوم عام ونريد إجبار الفئات المشتقة (**Derived Classes**) على تنفيذ تفاصيل معينة.

```
abstract class Animal
{
    public abstract void MakeSound();
}
```

4. Sealed Class (الفئة المغلقة)

المعنى: هي فئة نهائية ومغلقة، لا يمكن لأي فئة أخرى أن ترث منها (Cannot be inherited).

متى نستخدمها: لمنع التوريث (**Inheritance**) لأسباب أمنية أو لتجنب تغيير السلوك الأساسي للفئة (مثال: فئات الإعدادات أو الحماية).

```
sealed class SecurityConfig
{
    public string Token { get; set; }
}
```

5. Partial Class (الفئة الجزئية)

المعنى: تسمح بتقسيم كود الـ `Class` الواحدة عبر أكثر من ملف برمجي، وعند الترجمة (Compilation) يتم تجميعهم كفئة واحدة.

متى نستخدمها: مفيدة جداً عند العمل في فرق كبيرة، أو عند فصل الكود المنشأ تلقائياً (Auto-generated) عن الكود الذي يكتبه المبرمج (code).

```
// File 1
partial class Employee { public string Name { get; set; } }

// File 2
partial class Employee { public void Work() { } }
```

6. Generic Class (الفئة العامة)

المعنى: فئة لا ترتبط بنوع بيانات محدد مسبقاً، بل تستقبل نوع البيانات كعامل أو Parameter عند الاستخدام.

متى نستخدمها: لتحقيق أقصى قدر من الـ `Code Reusability` ولتجنب تكرار الكود مع ضمان الأمان من حيث نوع البيانات (`Type Safety`).

```
class Result<T>
{
    public T Data { get; set; }
}
```

عمرک سألت نفسك إيه هي أنواع الـ Classes في C# وإمتى نستخدم كل نوع في مشاريعنا؟ 🤔

لغة C# بتوفر لنا أدوات قوية جداً عشان نكتب كود نضيف ومنظم وقابل للتطوير (Scalable Software Design):
خلينا نبسط الموضوع.

- ♦ **Normal Class:** القالب الأساسي بتاعنا اللي ببنيني منه الـ Objects.
- ♦ **Static Class:** للـ Methods اللي مش محتاجة Object، زي العمليات الحسابية السريعة.
- ♦ **Abstract Class:** بنستخدمه كـ Base Class أساسي بييجبر الكلاسات التانية (Derived Classes) إنها تنفذ تفاصيل معينة.
- ♦ **Sealed Class:** لو عايز تحمي الكلاس بتاعك وتمنع الـ Inheritance منه تماماً، ده الحل!
- ♦ **Partial Class:** ممتاز لو عايز تقسم الكود بتاعك على أكثر من فايل عشان تنظم الشغل.
- ♦ **Generic Class:** البطل في تقليل تكرار الكود (Code Reusability) لأنه بيشتغل مع أي Data Type مع ضمان الـ Type Safety.

💡 في رأيك، إيه أكثر نوع بتستخدمه في شغلك اليومي كـ .NET Developer؟ وهل جربت تستفيد من الـ Sealed Classes؟
👉! في تحسين أداء الكود؟ شاركوني في التعليقات

#CSharp #DotNet #OOP #BackendDevelopment #SoftwareEngineering

Question 2: Generalization Concept Using Generics

مفهوم الـ **Generalization** (التعميم) باستخدام الـ **Generics** هو أحد أقوى ميزات لغة C# التي تساعدنا في بناء أنظمة مرنة.

ما هو الـ Generalization؟

التعميم يعني كتابة كود واحد يمكنه العمل مع مختلف أنواع البيانات (Data Types) دون الحاجة إلى تكرار الكود أو تخصيصه لنوع واحد فقط.

ما هي ال Generics في C#؟

ال **Generics** هي التقنية التي تسمح لنا بتأجيل تحديد نوع البيانات المستخدم في ال **Class** حتى وقت الإنشاء باستخدامها، يمكننا كتابة أكواد تعتمد على **Type Parameter** (Instantiation). بدلاً من نوع محدد (مثل `int` أو `string`).

كيف تساهم في كتابة كود قابل لإعادة الاستخدام (Code Reusability)؟

بدلاً من كتابة كلاس لتخزين النصوص، وكلاس آخر لتخزين الأرقام، نكتب **Generic Class** واحد يعالج كل الأنواع، مما يقلل من تكرار الأكواد (Code Duplication) ويزيد من جودة ال **Software Design**.

مفهوم ال Type Safety

على عكس استخدام نوع `object` الذي يتطلب عملية تحويل (Casting) وقد يؤدي لأخطاء في وقت التشغيل تضمن لنا ال **Generics** (Runtime Errors)، قوة وتأكد النوع (**Type Safety**) في وقت الترجمة (Compile time).

مثال عملي:

```
class Storage<T>
{
    public T Value { get; set; }
}

// Usage:
Storage<int> numberStorage = new Storage<int>();
numberStorage.Value = 10;

Storage<string> textStorage = new Storage<string>();
textStorage.Value = "Hello";
```

توضيح الكود: الرمز `T` هنا يعبر عن **Type Parameter**. عندما نقوم بإنشاء `numberStorage`، يقوم ال `Compiler` باستبدال `T` بالنوع `int`. وعندما ننشئ `textStorage`، يتم استبدال `T` بالنوع `string`. هذا يجعل نفس ال **Class** قادراً على العمل مع أكثر من نوع بكفاءة و **Type Safety** عالية.

Question 3: Hierarchy Design in Real Business

ما المقصود بـ Hierarchy Design؟

التصميم الهرمي للطبقات (**Class Hierarchy**) يعني ترتيب الكلاسات في هيكل تنظيمي يشبه الشجرة، حيث تنتقل الصفات والسلوكيات من الكلاسات العامة في الأعلى إلى الكلاسات المخصصة في الأسفل.

- **Base Class** : هو الفئة الأساسية (الأب) التي تحتوي على الخصائص والسلوكيات المشتركة (**Common behavior**).
- **Derived Class** : هو الفئة المشتقة (الابن) التي تكتسب الخصائص من الأب وتضيف عليها سلوكياتها الخاصة (**Specialized behavior**).
- **Inheritance** : التوريث هو الآلية التي تسمح بتطبيق هذا التصميم الهرمي وتسهيل الـ **Code Reusability**.

مثال تطبيقي في الشركات:

لنفترض أن لدينا نظام لشركة، يمكننا تمثيل الموظفين بهذا الهرم:

```
Employee
|
|-- Developer
|-- Manager
|-- Accountant
```

```
class Employee
{
    public string Name { get; set; }

    public void Work()
    {
        Console.WriteLine("Employee is working");
    }
}

class Developer : Employee
{
    public void WriteCode()
    {
        Console.WriteLine("Developer is writing code");
    }
}
```

الشرح: في هذا الكود، الـ **Derived Class** المسمى Developer يرث من الـ **Base Class** المسمى Employee باستخدام علامة **:**. هذا يعني أن المطور يمتلك تلقائياً خاصية الـ Name ودالة الـ Work() (وهي السلوك المشترك لجميع الموظفين)، وفي نفس الوقت يضيف سلوكه الخاص والمميز وهو الـ WriteCode().

استخدام هذا المفهوم في الأنظمة الواقعية:

- **(الأنظمة البنكية) Banking Systems** الـ **Base Class** يمكن أن يكون Account (يحتوي على الرصيد ودالة الإيداع)، وتتفرع منه فئات مثل SavingsAccount و CheckingAccount لإضافة قواعد فوائد وسحب مخصصة.
- **(أنظمة التجارة الإلكترونية) E-Commerce Systems** الـ **Base Class** يكون Product، ونجد فئات متفرعة مثل ElectronicProduct (يضيف فترة الضمان) و DigitalProduct (يضيف رابط التحميل).
- **(أنظمة الموارد البشرية) HR Systems** كما في مثالنا، Employee يتفرع منه أنواع مختلفة من الموظفين باختلاف طرق حساب رواتبهم وحوافزهم باستخدام الـ **Polymorphism**.
- **(أنظمة إدارة العيادات) Clinic Management Systems** فكرة الـ **Inheritance** ممتازة جداً عند تصميم هيكلية قواعد البيانات والـ Web APIs باستخدام أدوات مثل ASP.NET Core و Entity Framework، حيث يمكن أن يكون لدينا كلاس رئيسي، PersonCore، يحمل البيانات الأساسية (مثل الاسم ورقم الهاتف)، وتتفرع منه كلاسات Doctor و Patient لإضافة التخصص الطبي للمريض والسجلات المرضية للطبيب.